

SQL Scenario Based Questions

Legend: **SQL Keywords** | Other Code

Q: 1. You need to find all employees whose salary is greater than 50,000. Which SQL query would you use?

Ans:

```
SELECT *  
FROM employees  
WHERE salary > 50000;
```

Q: 2. A customer table contains duplicate email addresses. How would you identify duplicates?

Ans:

```
SELECT email, COUNT(*) AS count  
FROM customer  
GROUP BY email  
HAVING COUNT(*) > 1;
```

Q: 3. You accidentally inserted wrong data into a table. How would you correct it?

Ans:

```
1- UPDATE table_name  
   SET column_name = 'correct_value'  
   WHERE condition;  
  
2- UPDATE Employees  
   SET department = 'IT'  
   WHERE emp_id = 2;
```

Q: 4. A table contains NULL values in the phone number column. How would you retrieve only rows with missing phone numbers?

Ans:

```
SELECT *  
FROM table_name  
WHERE phone_number IS NULL;
```

Q: 5. You want to display employees sorted by highest salary first. Which clause would you use?

Ans:

```
SELECT *  
FROM employees  
ORDER BY salary DESC;
```

Q: 6. Two tables contain customer and order information. How would you retrieve customers along with their orders?

Ans:

```
SELECT c.customer_id, c.customer_name, o.order_id, o.order_date  
FROM customers c  
INNER JOIN orders o  
ON c.customer_id = o.customer_id;
```

Q: 7. You need to find customers who have not placed any orders. Which join would you use?

Ans:

```
SELECT c.customer_id, c.customer_name
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Q: 8. A manager asks for the total sales made by each employee. Which SQL function and clause would help?

Ans:

```
SELECT employee_id, SUM(sales_amount) AS total_sales
FROM sales
GROUP BY employee_id;
```

Q: 9. You want to count how many employees work in each department. How would you do it?

Ans:

```
SELECT department_id, COUNT(*) AS employee_count
FROM employees
GROUP BY department_id;
```

Q: 10. A report should show only departments with more than 10 employees. Which clause would you use after GROUP BY?

Ans:

```
SELECT department_id, COUNT(*) AS employee_count
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 10;
```

Q: 11. You need to fetch only the first 5 records from a table. How would you do it?

Ans:

```
SELECT *
FROM table_name
LIMIT 5;
```

Q: 12. An employee's department changed. How would you update only that employee's department?

Ans:

```
UPDATE employees
SET department_id = 5
WHERE employee_id = 101;
```

Q: 13. You want to remove all records from a table but keep the table structure. Which command would you use?

Ans:

```
TRUNCATE TABLE table_name;
```

Q: 14. A table should not allow duplicate employee IDs. Which constraint would you apply?

Ans:

```
ALTER TABLE employees
ADD CONSTRAINT pk_employee_id PRIMARY KEY (employee_id);
```

Q: 15. You need to ensure salary values cannot be NULL. Which constraint would you use?

Ans:

```
ALTER TABLE employees
    MODIFY salary DECIMAL(10,2) NOT NULL;
```

Q: 16. You want to combine results from two tables with the same structure and remove duplicates. Which operator would you use?

Ans:

UNION

Q: 17. A login system should store only unique usernames. How would you enforce this in SQL?

Ans:

```
ALTER TABLE users
    ADD CONSTRAINT uq_username UNIQUE (username);
```

Q: 18. You need to search for customers whose names start with "A". Which operator would you use?

Ans:

```
SELECT *
    FROM customers
    WHERE customer_name LIKE 'A%';
```

Q: 19. You want to find the second highest salary in an employee table. How would you approach it?

Ans:

```
SELECT MAX(salary)
    FROM employees
    WHERE salary < (SELECT MAX(salary) FROM employees);
```

Q: 20. A company wants faster search performance on employee_id column. What would you create?

Ans:

```
CREATE INDEX idx_employee_id
    ON employees(employee_id);
```

Q: 21. You need to list all departments that do not have any employees. Which SQL query would you use?

Ans:

```
SELECT d.department_id, d.department_name
    FROM departments d
    LEFT JOIN employees e ON d.department_id = e.department_id
    WHERE e.employee_id IS NULL;
```

Q: 22. You want to count the number of employees hired in each year. How would you do it?

Ans:

```
SELECT EXTRACT(YEAR FROM hire_date) AS hire_year,
    COUNT(*) AS employee_count
    FROM employees
    GROUP BY EXTRACT(YEAR FROM hire_date)
    ORDER BY hire_year;
```

Q: 23. You need to find employees who earn more than the average salary of their department. How would you write it?

Ans:

```
SELECT employee_id, first_name, salary, department_id
FROM employees e1
WHERE salary > (SELECT AVG(salary)
                FROM employees e2
                WHERE e1.department_id = e2.department_id);
```

Q: 24. You need to list every employee along with their manager's name. Which SQL query would you use?

Ans:

```
SELECT e.employee_id, e.first_name, e.last_name,
       m.first_name AS manager_first_name, m.last_name AS manager_last_name
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id;
```

Q: 25. You want to find the top 3 highest paid employees in each department. How would you do it?

Ans:

```
SELECT department_id, employee_id, first_name, salary
FROM (
    SELECT *, RANK() OVER (PARTITION BY department_id
                          ORDER BY salary DESC) AS rnk
    FROM employees
) t
WHERE rnk <= 3
ORDER BY department_id, salary DESC;
```

Q: 26. You need to find the total number of orders placed by each customer. How would you do it?

Ans:

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id;
```

Q: 27. You need to find the average order amount for each customer. How would you do it?

Ans:

```
SELECT customer_id, AVG(order_amount) AS avg_order_amount
FROM orders
GROUP BY customer_id;
```

Q: 28. You need to find customers who have placed more than 5 orders. Which SQL query would you use?

Ans:

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 5;
```

Q: 29. You want to find the total revenue generated each month. How would you do it?

Ans:

```

SELECT DATE_FORMAT(order_date, '%Y-%m') AS order_month,
       SUM(order_amount) AS total_revenue
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY order_month;

```

PostgreSQL

```

SELECT TO_CHAR(order_date, 'Mon-YY') AS order_month,
       SUM(total_amount) AS total_revenue
FROM orders
GROUP BY TO_CHAR(order_date, 'Mon-YY')
ORDER BY MIN(order_date);

```

Q: 30. You want to find the most recent order date for each customer. How would you do it?

Ans:

```

SELECT customer_id, MAX(order_date) AS last_order_date
FROM orders
GROUP BY customer_id;

```

Q: 31. You want to list all products that are out of stock. Which SQL query would you use?

Ans:

```

SELECT product_id, product_name
FROM products
WHERE stock_quantity = 0;

```

Q: 32. You need to find the number of active customers. How would you do it?

Ans:

```

SELECT COUNT(*) AS active_customer_count
FROM customers
WHERE status = 'Active';

```

Q: 33. You want to find customers who registered in the last 30 days. Which SQL query would you use?

Ans:

```

SELECT customer_id, customer_name, register_date
FROM customers
WHERE register_date >= CURRENT_DATE - INTERVAL '30' DAY;

```

Q: 34. You need to list orders along with customer names. Which SQL query would you use?

Ans:

```

SELECT o.order_id, o.order_date, c.customer_name
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;

```

Q: 35. You want to find the top 5 best-selling products based on total quantity sold. How would you do it?

Ans:

```

SELECT product_id, product_name, SUM(quantity) AS total_quantity_sold
FROM order_items
GROUP BY product_id, product_name
ORDER BY total_quantity_sold DESC

```

LIMIT 5;

Q: 36. You need to find the total quantity sold of each product in each month. How would you do it?

Ans:

```
SELECT product_id, DATE_FORMAT(order_date, '%Y-%m') AS order_month,  
        SUM(quantity) AS total_quantity_sold  
FROM order_items  
GROUP BY product_id, DATE_FORMAT(order_date, '%Y-%m')  
ORDER BY product_id, order_month;
```

PostgreSQL

```
SELECT  
    oi.product_id,  
    TO_CHAR(DATE_TRUNC('month', o.order_date), 'Mon-YYYY') AS sales_month,  
    SUM(oi.quantity) AS total_quantity_sold  
FROM orders o  
JOIN order_items oi  
    ON o.order_id = oi.order_id  
GROUP BY  
    oi.product_id,  
    DATE_TRUNC('month', o.order_date)  
ORDER BY  
    DATE_TRUNC('month', o.order_date),  
    oi.product_id;
```

Q: 37. You want to find the average rating of each product. Which SQL query would you use?

Ans:

```
SELECT product_id, AVG(rating) AS average_rating  
FROM product_reviews  
GROUP BY product_id  
ORDER BY average_rating DESC;
```

Q: 38. You need to find products that have never been ordered. Which SQL query would you use?

Ans:

```
SELECT p.product_id, p.product_name  
FROM products p  
LEFT JOIN order_items oi ON p.product_id = oi.product_id  
WHERE oi.product_id IS NULL;
```

Q: 39. You want to find customers who have spent more than \$1000 in total. How would you do it?

Ans:

```
SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent  
FROM customers c  
JOIN orders o ON c.customer_id = o.customer_id  
GROUP BY c.customer_id, c.customer_name  
HAVING SUM(o.total_amount) > 1000  
ORDER BY total_spent DESC;
```

Q: 40. You need to find the category that has the highest total sales. How would you do it?

Ans:

```

SELECT p.category_id, c.category_name, SUM(oi.quantity * oi.unit_price) AS
total_sales
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
JOIN categories c ON p.category_id = c.category_id
GROUP BY p.category_id, c.category_name
ORDER BY total_sales DESC
LIMIT 1;

```

Q: 41. You need to find the second highest salary in the company. How would you do it?

Ans:

```

SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

```

Q: 42. You want to find duplicate email addresses in the customers table. How would you do it?

Ans:

```

SELECT email, COUNT(*) AS email_count
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;

```

Q: 43. You need to list employees who joined in the same month as their manager. How would you do it?

Ans:

MySQL

```

SELECT e.employee_id, e.first_name, e.last_name, m.first_name AS
manager_first_name,
m.last_name AS manager_last_name
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id
WHERE DATE_FORMAT(e.hire_date, '%Y-%m') = DATE_FORMAT(m.hire_date, '%Y-%m');

```

PostgreSQL version:

```

SELECT
e.employee_id,
e.first_name,
e.last_name,
e.hire_date AS emp_hire_date,
m.first_name AS manager_first_name,
m.last_name AS manager_last_name,
m.hire_date AS manager_hire_date
FROM employees e
JOIN employees m
ON e.manager_id = m.employee_id
WHERE TO_CHAR(e.hire_date, 'YYYY-MM') = TO_CHAR(m.hire_date, 'YYYY-MM');

```

Q: 44. You want to find the department(s) where the average salary is greater than the company average salary. How would you do it?

Ans:

```

SELECT d.department_id, d.department_name, AVG(e.salary) AS avg_salary
FROM departments d
JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
HAVING AVG(e.salary) > (SELECT AVG(salary) FROM employees);

```

Q: 45. You need to list customers who have placed orders in all categories available in the system. How would you do it?

Ans:

```

SELECT c.customer_id, c.customer_name
FROM customers c
WHERE NOT EXISTS (
    SELECT category_id FROM categories
    EXCEPT
    SELECT DISTINCT p.category_id
    FROM orders o
    JOIN products p ON o.product_id = p.product_id
    WHERE o.customer_id = c.customer_id
);

```

Q: 46. You want to find the customer who has placed the most orders. How would you do it?

Ans:

```

SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS order_count
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY order_count DESC
LIMIT 1;

```

Q: 47. You need to find the month in which the highest number of employees were hired. How would you do it?

Ans:

```

SELECT DATE_FORMAT(hire_date, '%Y-%m') AS hire_month,
COUNT(*) AS hire_count
FROM employees
GROUP BY DATE_FORMAT(hire_date, '%Y-%m')
ORDER BY hire_count DESC
LIMIT 1;

```

Q: 48. You want to find the total number of employees in each department, including departments that have no employees. How would you do it?

Ans:

```

SELECT d.department_id, d.department_name, COUNT(e.employee_id) AS
employee_count
FROM departments d
LEFT JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
ORDER BY d.department_id;

```

Q: 49. You need to list the top 5 customers who have spent the most in total. How would you do it?

Ans:

```

SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent
FROM customers c

```



```

JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_spent DESC
LIMIT 5;

```

Q: 50. You want to find departments that have both more than 5 employees and an average salary greater than 60000. How would you do it?

Ans:

```

SELECT d.department_id, d.department_name,
       COUNT(e.employee_id) AS emp_count, AVG(e.salary) AS avg_salary
FROM departments d
JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
HAVING COUNT(e.employee_id) > 5 AND AVG(e.salary) > 60000
ORDER BY avg_salary DESC;

```

★ BONUS QUESTIONS (ADVANCED SQL)

Q: B1. You need to find the second highest order amount from the orders table. How would you do it?

Ans:

```

SELECT MAX(order_amount) AS second_highest_amount
FROM orders
WHERE order_amount < (SELECT MAX(order_amount) FROM orders);

```

Q: B2. You want to find customers who have not placed any orders yet. How would you do it?

Ans:

```

SELECT c.customer_id, c.customer_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

```

Q: B3. You want to find the lifetime value (total spent) of each customer, including customers who have never placed an order. How would you do it?

Ans:

```

SELECT c.customer_id, c.customer_name, COALESCE(SUM(o.total_amount), 0) AS
lifetime_value
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY lifetime_value DESC;

```

Q: B4. You want to find the running total of sales (total_amount) over time for all orders. How would you do it?

Ans:

```

SELECT order_date, total_amount,
       SUM(total_amount) OVER (ORDER BY order_date) AS running_total
FROM orders
ORDER BY order_date;

```

Q: B5. You want to find the top 3 products in each category by total quantity sold. How would you do it?

Ans:

```
WITH ranked_products AS (  
    SELECT p.category_id, p.product_id, p.product_name,  
           SUM(oi.quantity) AS total_quantity,  
           ROW_NUMBER() OVER (PARTITION BY p.category_id ORDER BY  
SUM(oi.quantity) DESC) AS rn  
    FROM products p  
    JOIN order_items oi ON p.product_id = oi.product_id  
    GROUP BY p.category_id, p.product_id, p.product_name  
)  
SELECT category_id, product_id, product_name, total_quantity  
FROM ranked_products  
WHERE rn <= 3  
ORDER BY category_id;
```

```
INSERT INTO table_name (column1, column2, column3)  
VALUES (value1, value2, value3);
```

```
INSERT INTO employees (employee_id, first_name, salary)  
VALUES  
(101, 'Amit', 50000),  
(102, 'Ravi', 60000),  
(103, 'Neha', 70000);
```

```
CREATE TABLE employees (  
    employee_id INT,  
    first_name VARCHAR(50),  
    salary INT  
);
```

```
DELETE FROM table_name  
WHERE condition;
```

```
DELETE FROM employees  
WHERE employee_id = 103;
```